

Network Security Projects

Contents

0.1	Project 1B: Intelligent Protocol Analyzer & Lightweight Intrusion Detection System	2
0.1.1	Project Overview	2
0.1.2	Problem Statement	2
0.1.3	Proposed Solution	2
0.1.4	Objectives	2
0.1.5	System Modules	3
0.1.6	Packet Capture Engine	3
0.1.7	Packet Parser	4
0.1.8	Session Manager	4
0.1.9	TCP Stream Reassembly	5
0.1.10	HTTP Protocol Analyzer	5
0.1.11	DNS Analyzer	6
0.1.12	TLS ClientHello Parser	6
0.1.13	ARP Spoof Detection Module	7
0.1.14	Port Scan Detection	7
0.1.15	Rule-Based Detection Engine	8
0.1.16	Alert Management System	9
0.1.17	Database Design	9
0.1.18	REST API	10
0.1.19	Live Dashboard	10
0.1.20	Technology Stack	11
0.1.21	Core Networking Concepts	11
0.1.22	Project Deliverables	12
0.1.23	Learning Outcomes	12
0.1.24	Future Enhancements	13
0.1.25	Project Summary	13

0.1 Project 1B: Intelligent Protocol Analyzer & Lightweight Intrusion Detection System

0.1.1 Project Overview

The Intelligent Protocol Analyzer extends the Beginner Packet Sniffer into a stateful network monitoring and intrusion detection system. Instead of analyzing individual packets independently, the system reconstructs complete communication sessions, extracts application-layer metadata, detects suspicious behaviour through rule-based analysis, and provides real-time visualization of network activities.

The objective of this project is to understand how enterprise Intrusion Detection Systems such as Snort, Suricata and Zeek analyze traffic beyond individual packets.

0.1.2 Problem Statement

Traditional packet sniffers capture packets independently and display packet headers without understanding the relationship between multiple packets. Most cyber attacks including TCP SYN scans, ARP spoofing, DNS tunneling, credential attacks and session hijacking cannot be detected by inspecting a single packet.

A security monitoring platform must reconstruct network sessions, extract meaningful protocol information and continuously monitor traffic behaviour to identify malicious activities.

Therefore, the objective is to design and implement a lightweight Intrusion Detection System capable of performing stateful packet analysis, protocol parsing, anomaly detection and live monitoring.

0.1.3 Proposed Solution

The proposed solution converts the packet sniffer into a modular protocol analysis engine.

Captured packets first pass through a packet parser where Ethernet, IPv4, TCP, UDP and ARP headers are decoded.

Packets are then grouped into communication sessions using a Session Manager.

TCP streams are reconstructed to obtain complete application-layer payloads.

Application protocols such as HTTP, DNS and TLS are analyzed to extract metadata.

Finally, a Rule Engine evaluates traffic behaviour and generates alerts whenever suspicious patterns are detected.

All traffic flows and generated alerts are stored inside a SQLite database and displayed using a real-time dashboard.

0.1.4 Objectives

- Understand stateful packet inspection.
- Learn TCP stream reconstruction.
- Implement application layer protocol parsing.
- Build a lightweight rule engine.
- Detect common network attacks.

- Store network events inside a database.
- Visualize live network traffic.
- Build a modular cybersecurity application.

0.1.5 System Modules

The entire system is divided into multiple independent modules.

1. Packet Capture Engine
2. Packet Parser
3. Session Manager
4. TCP Stream Reassembly
5. HTTP Analyzer
6. DNS Analyzer
7. TLS Metadata Extractor
8. ARP Monitoring Module
9. Rule Engine
10. Alert Manager
11. SQLite Database
12. REST API
13. Live Dashboard

0.1.6 Packet Capture Engine

The Packet Capture Engine continuously captures packets from a selected network interface using Scapy.

Responsibilities include

- Capturing live traffic
- Selecting network interface
- Applying Berkeley Packet Filters
- Sending packets to the parser
- Exporting traffic into PCAP files

0.1.7 Packet Parser

The parser manually extracts protocol headers instead of depending upon Scapy's summary output.

The following protocol headers are decoded.

- Ethernet
- IPv4
- IPv6
- TCP
- UDP
- ICMP
- ARP

Extracted fields include

- Source MAC Address
- Destination MAC Address
- Source IP Address
- Destination IP Address
- Source Port
- Destination Port
- Sequence Number
- Acknowledgement Number
- Window Size
- TTL
- Protocol

0.1.8 Session Manager

The Session Manager groups packets into communication sessions.

Each session is uniquely identified using

(Source IP, Destination IP, Source Port, Destination Port, Protocol)

The Session Manager stores

- Session Start Time

- Last Activity
- Total Packets
- Total Bytes
- Connection Duration
- Retransmissions
- Connection State

0.1.9 TCP Stream Reassembly

Many application-layer protocols transmit information across multiple TCP packets. The reassembly engine reconstructs complete streams using

- Sequence Numbers
- ACK Numbers
- Window Size
- Payload Offset
- Missing Segment Detection

After reconstruction the analyzer can recover complete

- HTTP Requests
- HTTP Responses
- SMTP Messages
- FTP Commands
- TLS ClientHello Packets

0.1.10 HTTP Protocol Analyzer

The HTTP parser extracts

- Request Method
- URI
- Host
- User Agent
- Cookie
- Status Code

- Content Length
- MIME Type

The extracted metadata is useful for

- Web Traffic Monitoring
- Malicious URL Detection
- User Behaviour Analysis
- Session Investigation

0.1.11 DNS Analyzer

The DNS parser extracts

- Query Name
- Query Type
- Response
- TTL
- Resolver
- Record Type

To identify DNS tunneling attacks, Shannon Entropy is computed for every queried subdomain.

Large entropy values indicate randomly generated domains commonly used during data exfiltration.

0.1.12 TLS ClientHello Parser

Encrypted HTTPS traffic cannot be decrypted.

Instead, metadata is extracted from TLS ClientHello packets.

Extracted information includes

- TLS Version
- Supported Cipher Suites
- Supported Extensions
- Server Name Indication
- Supported Groups
- Signature Algorithms
- JA3 Fingerprint

JA3 fingerprints help identify malicious clients without decrypting traffic.

0.1.13 ARP Spoof Detection Module

The ARP monitoring module continuously observes Address Resolution Protocol traffic flowing through the local network.

Every observed mapping between an IP address and a MAC address is stored inside an ARP cache.

Whenever a known IP address suddenly resolves to a different MAC address, the system raises an ARP Spoofing alert.

The module maintains

- Current ARP Table
- Historical ARP Table
- MAC Address Changes
- Number of ARP Replies
- Duplicate IP Detection

Generated alerts include

- ARP Cache Poisoning
- Duplicate Gateway Detection
- Suspicious Gratuitous ARP
- MAC Address Hijacking

0.1.14 Port Scan Detection

One packet alone cannot determine whether a port scan is occurring.

The detection engine therefore maintains a sliding observation window.

For every source IP address, the following information is stored.

- Number of SYN Packets
- Number of Destination Ports Contacted
- Scan Duration
- Scan Frequency
- Connection Success Rate

If the number of unique destination ports exceeds a predefined threshold within a short duration, the engine generates a Port Scan alert.

Supported scan detections include

- TCP SYN Scan
- TCP Connect Scan

- FIN Scan
- NULL Scan
- Xmas Scan
- UDP Scan

0.1.15 Rule-Based Detection Engine

The detection engine evaluates every parsed packet and every reconstructed flow against a predefined set of security rules.

Rules are stored in YAML format allowing administrators to modify detection logic without recompiling the application.

Each rule contains

- Rule ID
- Rule Name
- Protocol
- Source IP
- Destination IP
- Source Port
- Destination Port
- Detection Threshold
- Severity
- Alert Message

The rule engine supports

- Exact Matching
- Wildcard Matching
- Time-based Detection
- Frequency Detection
- Threshold-based Alerts
- Custom Detection Rules

0.1.16 Alert Management System

Whenever a detection rule is triggered, an alert object is generated.

Each alert contains

- Alert ID
- Timestamp
- Severity
- Source Address
- Destination Address
- Protocol
- Attack Category
- Detection Rule
- Description
- Supporting Evidence

Alerts are categorized into

- Information
- Low
- Medium
- High
- Critical

0.1.17 Database Design

SQLite is used to persist captured traffic, session information and generated alerts.

The database consists of the following tables.

- Packets
- Sessions
- DNS Queries
- HTTP Requests
- TLS Metadata
- Alerts
- Detection Rules
- Statistics

Each alert references the session responsible for triggering the detection.

0.1.18 REST API

The application exposes a REST API allowing external applications to retrieve captured traffic and generated alerts.

The following endpoints are implemented.

- GET /packets
- GET /sessions
- GET /alerts
- GET /statistics
- GET /dns
- GET /http
- GET /tls
- POST /rules
- DELETE /alerts

The API returns JSON responses making integration with dashboards and SIEM platforms straightforward.

0.1.19 Live Dashboard

A browser-based monitoring dashboard provides real-time visualization of network activity.

Dashboard components include

- Live Packet Counter
- Active Sessions
- Network Throughput
- Protocol Distribution
- HTTP Requests
- DNS Queries
- TLS Connections
- Recent Alerts
- Top Talkers
- Top Destination Ports
- Packet Rate Graph
- Alert Timeline

The dashboard receives updates using WebSockets without requiring manual page refreshes.

0.1.20 Technology Stack

- Python 3
- Scapy
- dpkt
- FastAPI
- Flask
- SQLite
- SQLAlchemy
- WebSockets
- Chart.js
- PyYAML
- Rich
- Git

0.1.21 Core Networking Concepts

The project reinforces several networking concepts.

- OSI Model
- TCP/IP Stack
- Stateful Inspection
- TCP State Machine
- Sliding Window Protocol
- TCP Stream Reassembly
- HTTP Protocol
- DNS Protocol
- TLS Handshake
- JA3 Fingerprinting
- ARP Resolution
- Intrusion Detection Systems
- Rule-Based Detection
- Event Correlation

0.1.22 Project Deliverables

Upon successful completion, the project should provide

- Stateful Packet Analyzer
- TCP Session Reassembly Engine
- HTTP Protocol Analyzer
- DNS Analyzer
- TLS Metadata Extractor
- ARP Spoof Detector
- Port Scan Detector
- YAML Rule Engine
- SQLite Database
- REST API
- Live Monitoring Dashboard
- Alert Logging System

0.1.23 Learning Outcomes

After completing this project, students will be able to

- Build a modular network analysis application.
- Understand the difference between packet-based and flow-based analysis.
- Implement session tracking.
- Design custom IDS detection rules.
- Parse multiple application-layer protocols.
- Develop live monitoring dashboards.
- Store and analyze network events.
- Understand how enterprise IDS platforms operate.

0.1.24 Future Enhancements

The project can be extended by adding

- Threat Intelligence Integration
- AbuseIPDB Lookup
- VirusTotal API
- GeoIP Mapping
- Email Alerting
- Slack Notifications
- Syslog Export
- SIEM Integration
- Multi-threaded Packet Capture
- Docker Deployment
- Cloud-based Monitoring
- Role-Based Access Control

0.1.25 Project Summary

This project transforms a simple packet sniffer into a lightweight Intrusion Detection System capable of understanding communication sessions rather than isolated packets.

The completed system performs packet capture, protocol parsing, TCP stream reconstruction, rule-based attack detection, database logging, REST API services and real-time visualization.

It establishes the foundation required for building a distributed enterprise-grade Network Detection and Response platform, which is implemented in the Advanced level project.